

Spec-Driven Development

Metodologia, Configurazione e Best Practices

*Dal prompt alla specifica, dalla specifica al codice:
un processo controllato, iterativo e riproducibile.*

Agenda



01

Cos'è lo Spec-Driven Development

Principi fondamentali e vantaggi



02

Evoluzione della Metodologia SDD

Dal Planning Mode al Brainstorming avanzato



03

Workflow SDD Tradizionale

Planning Mode: prompt → piano di sviluppo → esecuzione → review



04

Workflow SDD Avanzato

Brainstorming & Specification: esplorazione → specifica → piano di sviluppo → subagent (esecuzione e review)



05

Confronto tra i Workflow SDD

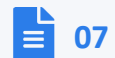
Quando usare quale approccio e perchè



06

Context Engineering & Best Practices

Principi di ingegneria del contesto per l'AI



07

Il file CLAUDE.md

Ruolo, struttura e workflow di generazione (Lite, Intermediate, Advanced)



08

Prompt Templates per lo SDD

Template strutturati per ogni fase del workflow



09

Lezioni Apprese e Key Takeaways

Sintesi operativa e linee guida pratiche

01

Cos'è lo Spec-Driven Development?

Principi fondamentali, vantaggi e contesto

Cos'è lo Spec-Driven Development?

Una metodologia di sviluppo in cui l'AI non scrive codice liberamente, ma implementa una specifica dettagliata, approvata dall'utente, attraverso un ciclo controllato di pianificazione, esecuzione e revisione.



Specifica Prima

Si definisce il COSA prima
di chiedere il COME



Controllo Umano

L'utente approva ogni fase
prima di procedere



Qualità Prevedibile

Risultati ripetibili, non
dipendenti dal caso

Consistenza

Stesso input = stesso output
prevedibile

Riproducibilità

Rifare il progetto da zero è possibile

Manutenibilità

Spec aggiornata = codice aggiornato

Onboarding

Nuovi sviluppatori leggono la spec

SDD vs Vibe Coding

Lo SDD trasforma l'AI da "assistente casuale" a "sviluppatore guidato da specifiche"

Aspetto	Vibe Coding	Spec-Driven Development
Approccio	"Fai qualcosa" → correggi	Specifica → Piano di sviluppo → Esecuzione → Review
Controllo	L'AI decide cosa fare	L'utente approva ogni fase
Risultato	Imprevedibile, variabile	Riproducibile, prevedibile
Iterazioni	Molte correzioni post-hoc	Poche / nessuna correzione
Scalabilità	Non scalabile	Scalabile, documentato
Copertura AI stimata	40-60%	90-95%+

"Il dettaglio in più nella specifica fa la differenza tra 'costruisci qualcosa' e 'costruisci esattamente questo'."

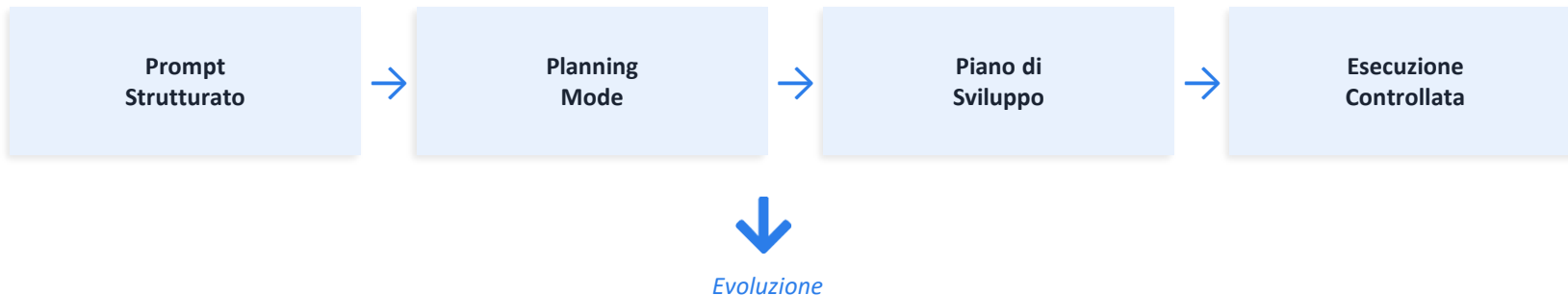
02

Evoluzione della Metodologia SDD

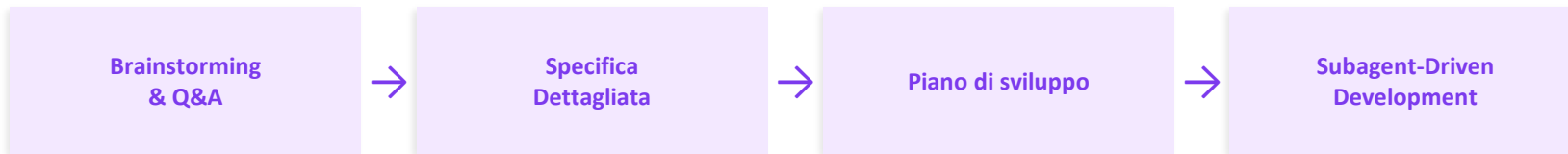
Dal Planning Mode al Brainstorming & Specification

Evoluzione della Metodologia SDD

WORKFLOW TRADIZIONALE (Planning Mode)



WORKFLOW AVANZATO (Brainstorming & Specification)



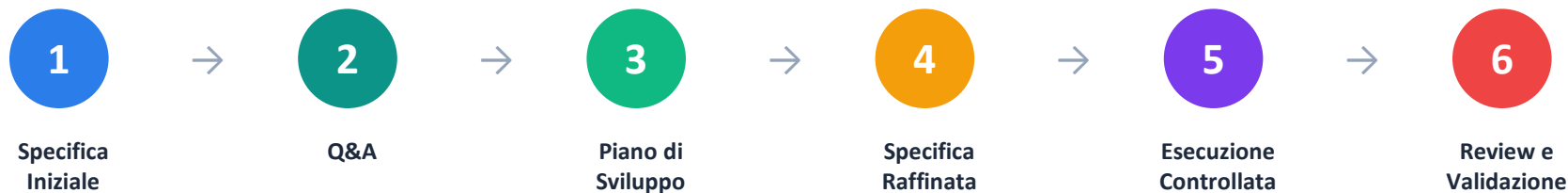
Il workflow avanzato introduce una fase di esplorazione strutturata (brainstorming) prima della specifica, producendo specifiche più complete e riducendo le iterazioni correttive.

03

Workflow SDD Tradizionale

Planning Mode: dal prompt strutturato all'implementazione controllata

Il Workflow SDD Tradizionale: 6 Fasi



Ogni fase richiede approvazione prima di procedere. Il ciclo può essere iterato: se la review identifica problemi, si torna alla fase appropriata.



Fase 1: Specifica Iniziale

Prompt strutturato con obiettivo, requisiti, contesto, vincoli. Richiede esplicitamente una fase di planning.



Fase 2: Q&A

Claude identifica ambiguità e propone opzioni concrete. Colma i gap PRIMA di scrivere codice.



Fase 3: Piano di Sviluppo

Schema DB, data layer, API, UI dettagliati. Piano salvato come artifact (development-plan.md).



Fase 4: Specifica Raffinata

Prompt aggiornato con le decisioni del Q&A. La specifica cresce in dettaglio e precisione.



Fase 5: Esecuzione Controllata

Implementazione su Claude Code con revisione diff-by-diff. Riferimento al piano approvato.



Fase 6: Review e Validazione

Verifica post-implementazione con specifica come benchmark di conformità.

Caso Pratico: EV-002 Configurable Working Hours

Workflow SDD Tradizionale (Planning Mode)

Obiettivo: permettere la configurazione delle ore lavorative giornaliere per ogni risorsa, con un modello a 3 livelli (default, pattern settimanale, eccezioni), e gestire assenze con granularità di 30 minuti.



- 1 Specifica iniziale**
Prompt con obiettivo, requisiti principali, planning mode richiesto. Buona pratica ma troppo vago su aspetti critici.
- 2 Q&A con Claude**
Claude identifica 5+ ambiguità. Decisioni su: scope settimanale, festività, DailyWorkingHours, approccio architetturale.
- 3 Piano di Sviluppo**
Documento di 340 righe: SQL completi, confronto 2 approcci architetturali, specifiche UI. Salvato come development-plan.md.
- 4 Specifica Raffinata**
Prompt aggiornato: 3-tier esplicito, Override > Weekly > Default, range 0.0-24.0, festività full-day.
- 5 Esecuzione**
Implementazione in Claude Code con riferimento al piano approvato. Revisione diff-by-diff.
- 6 Review**
Verifica conformità: funzionalità rotte, mancanti, possibili miglioramenti. Ciclo iterativo se necessario.

Specifica Iniziale

Il prompt originale includeva:

- **Obiettivo chiaro**

Configurare ore lavorative giornaliere per risorsa

- **Requisiti principali**

DailyWorkingHours, variazioni giorno per giorno, assenze 30 min

- **Vincoli tecnici**

Script SQL, niente unit test, persistenza DB

- **Riferimenti al contesto**

development-plan.md, analisi funzionale

- **Planning phase richiesta**

Approvazione obbligatoria prima del coding



Buona pratica

Richiedere esplicitamente una fase di planning prima dell'implementazione evita che l'AI "parta in quarta".



Area di miglioramento

Troppo vago su aspetti critici: modello a 3 livelli, migrazione assenze, integrazione festività, UI.

Fase 2

Q&A e revisione della specifica

Q1

Scope settimanale?

Full week Mon-Sun

Q2

Posizionamento UI?

Nuovo tab nella pagina Resource

Q3

Range DailyWorkingHours?

0.0 - 24.0 (da specifica task)

Q4

Festività parziali?

No, solo giornata intera

Claude ha identificato ambiguità nella specifica e ha proposto opzioni concrete, permettendo decisioni informate. Questo è il valore della fase Q&A: colmare i gap PRIMA di scrivere codice.

Piano di Sviluppo

Schema Database

4 entità (Resource, WeeklySchedule, ScheduleOverride, Absence), SQL completo

Data Layer

Modelli C#, ViewModels, Repository con Dapper + stored procedure

API & Controller

Web API + MVC Controllers, AutoMapper, validazione FluentValidation

UI Design

Grids Kendo, editor numerici, localizzazione italiana completa

Output del Piano di Sviluppo

Il piano prodotto da Claude ha dettagliato: SQL completi per ogni tabella, confronto tra 2 approcci architetturali (con pro/contro e raccomandazione), metodi per Repository e Controller, specifiche per ogni grid UI. Il documento di 340 righe è stato prodotto nella stessa sessione di Q&A e salvato come development-plan.md.

Specifica Raffinata

Il prompt originale è stato aggiornato incorporando le decisioni prese durante il Q&A.

PRIMA

"allow to configure and to manage daily working hours day-by-day variations, like managing a real time-resource scheduling calendar"

Nessuna menzione di:

- Modello a 3 livelli di risoluzione
- Migrazione da date-range a per-day
- Integrazione festività
- Specifiche UI e labels italiane
- Cascade DailyWorkingHours



DOPO

Requisiti specifici aggiuntivi:

- Full week Mon-Sun, Sat/Sun default 0h
- 3-tier: Override > Weekly > Default
- Range 0.0 - 24.0
- Festività full-day (AbsenceHours = NULL)

Ma ancora mancano:

- Dettagli di migrazione dati
- Specifiche UI complete
- Business rules per festività

Esecuzione e Review



Fase 5: Esecuzione

Il prompt di esecuzione includeva:

- La specifica raffinata completa
- Riferimento al development-plan.md
- Istruzione di leggere il piano
- Istruzione di iniziare l'implementazione

Nota: la fase di planning è stata rimossa
dal prompt, perché non serviva più.

L'implementazione avviene su Claude Code
con revisione diff-by-diff.



Fase 6: Review

Il prompt di review chiedeva di:

- Identificare funzionalità rotte
- Identificare funzionalità mancanti
- Valutare possibili miglioramenti

Il prompt ripeteva la specifica completa
per dare a Claude il contesto necessario a
valutare la conformità dell'implementazione.

*Il ciclo si chiude: la review può generare
nuovi task di correzione (nuove iterazioni).*

Scelte corrette



Workflow strutturato in fasi

Separazione netta tra planning, Q&A, piano, esecuzione e review.



Q&A prima dell'implementazione

Claude ha chiesto 5 domande + scelta architettuale, evitando assunzioni.



Approvazione esplicita a ogni gate

Il piano non è stato eseguito senza conferma: ogni fase aveva un checkpoint.



Piano salvato come artifact

development-plan.md persistente, riutilizzabile in sessioni successive.



Review come fase separata

Verifica post-implementazione con specifica come benchmark di conformità.



Iterazione sul design durante Q&A

Schema DB rivisto con feedback (rimozione campi inutili, semplificazione Absence).

Weak points



Specifica iniziale troppo vaga

Concetti chiave come il modello a 3 livelli e la migrazione dati assenti. Claude avrebbe raggiunto solo il 60-70% del risultato.



Troppa delega al contesto esterno

"Read development-plan.md" scarica la comprensione su Claude. I requisiti essenziali devono essere nel prompt.



Nessuna specifica UI

Layout, labels italiane, comportamento editor, messaggi di errore: tutti mancanti nel prompt originale.



Mix di livelli di astrazione

Decisioni architetturali e vincoli di processo ("niente unit test") nella stessa lista: confusione.



Business rules implicite

Integrazione festività, cascade DailyWorkingHours, assenze duplicate permesse: nessuna menzionata.



No schema dati esplicito

Anche senza codice, specificare le entità e i loro campi avrebbe eliminato ambiguità.

L'Impatto della Specifica

Aspetto	Prompt Originale	Specifica Ottimizzata
Lunghezza	~35 righe	~150 righe
Requisiti funzionali	3 generici	7 dettagliati (FR-1 ... FR-7)
Schema dati	Assente	4 entità con campi e vincoli
Specifiche UI	Assenti	Labels, editor, layout, messaggi
Business rules	Implicite	Esplicite (festività, cascade, ...)
Stima copertura AI	60-70%	~95%
Iterazioni correttive	Molte	Poche / nessuna



La qualità del risultato è proporzionale alla qualità della specifica.
Ogni ora di planning ne risparmia tre di correzioni.

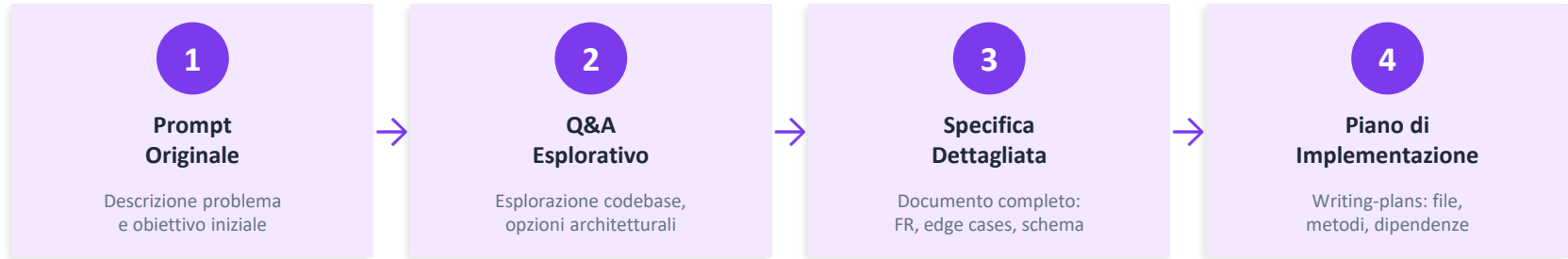
04

Workflow SDD Avanzato

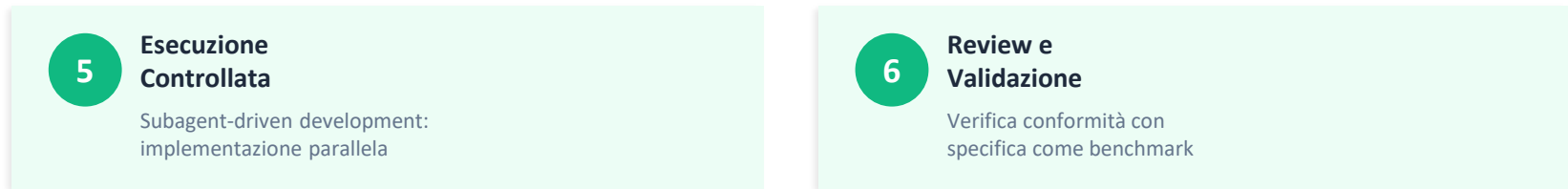
Brainstorming & Specification: esplorazione guidata prima dell'implementazione

Workflow SDD Avanzato: Brainstorming & Specification

FASE DI BRAINSTORMING (su Claude Desktop o Claude Code)



FASE DI ESECUZIONE (su Claude Code)



Differenza chiave: la fase di brainstorming esplora il problema PRIMA della specifica. Il Q&A è più profondo, l'architettura viene definita in modo collaborativo, e il piano di sviluppo risulta maggiormente dettagliato.

Caso Pratico: EV-003 Modified Gantt Date Calculation

Workflow SDD Avanzato (Brainstorming & Specification)



Il Problema

Il Gantt assume che tutte le risorse lavorino 8h/giorno. Un task di 16h viene mostrato come 2 giorni per tutti. Ma una risorsa part-time da 4h/giorno ha bisogno di 4 giorni, non 2. Questo causa timeline non realistiche.

Decisioni chiave emerse dal Brainstorming:

- ✓ **Unità di calcolo:** Mantenere giorni (1 giorno = 8h, conversione interna)
- ✓ **Ricalcolo:** Solo manuale (nessun auto-trigger su CRUD)
- ✓ **Architettura:** Nuova classe ResourceCapacityUtilities condivisa
- ✓ **Due path di calcolo:** Core condiviso per TaskManagement e WorkAssignment
- ✓ **Algoritmo:** 3-tier resolution, pre-fetch dati, walk day-by-day
- ✓ **Codice obsoleto:** Eliminazione DateTimeUtilities + 15 metodi inutilizzati

1

Nuovo file

4

File modificati

1

File eliminato

15

Metodi rimossi

0

Modifiche SQL

Lezioni dal Prompt Review (EV-003)

Anche con un brainstorming approfondito, la specifica può avere gap critici:



Boundary MVC-WebApi non specificato

La specifica mostra dipendenza diretta, ma MVC non può referenziare WebApi. Serviva un Controller HTTP + batch model.



Batch processing non previsto

La specifica implica chiamate HTTP per-activity. L'implementazione ha dovuto aggregare per risorsa.



Safety guard insufficiente

365 giorni inadeguati per scenari estremi (2h/giorno, 3 giorni/settimana). Implementazione usa 730.



EndDate esclusivo non documentato

Convenzione critica: la data di fine è "ultimo giorno lavorativo + 1" per Kendo Gantt. Mai menzionato.



Festività per unassigned mancanti

La specifica dice "8h/giorno Mon-Fri" ma non menziona lo skip delle festività.



3 frasi che avrebbero cambiato l'esito: confini architetturali, convenzione EndDate, festività per unassigned.

05

Confronto tra i Workflow SDD

Planning Mode vs Brainstorming & Specification

Confronto: Planning Mode vs Brainstorming & Specification

Aspetto	Planning Mode (Tradizionale)	Brainstorming & Specification (Avanzato)
Fase iniziale	Prompt strutturato diretto	Brainstorming esplorativo con Q&A approfondito
Specifica	Evolve dal prompt dopo Q&A	Documento formale completo con FR, edge cases, schema
Piano di sviluppo	Generato in planning mode	Writing-plans: file-by-file, metodo-by-metodo
Esecuzione	Diff-by-diff controllata	Subagent-driven development (parallela)
Complessità problema	Problema ben definito	Problema che richiede esplorazione
Strumenti upstream	Claude Desktop per analisi iniziale	Claude Desktop/Code per brainstorming + analisi codebase
Profondità analisi	Q&A con 5+ domande	Esplorazione codebase, scoperta path multipli, decisioni architetturali
Ideale per	Feature note, campi noti	Algoritmi, integrazioni, refactoring cross-layer

Quando Usare Quale Workflow?

PLANNING MODE

- Il problema è ben definito e chiaro
- L'architettura è già nota
- Feature standard (CRUD, UI, reports)
- Requisiti chiari fin dall'inizio
- Scope limitato (pochi file coinvolti)
- Urgenza: serve velocità di delivery

BRAINSTORMING & SPECIFICATION

- Il problema richiede esplorazione
- Architettura da scoprire/decidere
- Algoritmi complessi, refactoring cross-layer
- Integrazioni tra layer/servizi
- Scope ampio (molti file, dipendenze)
- La qualità della specifica e del risultato sono prioritari

I workflow sono complementari, non mutuamente esclusivi. Si può iniziare con Planning Mode e passare a Brainstorming se emergono complessità impreviste.

Quale Strumento per Quale Fase?

Claude Desktop

Fasi upstream (pensare, pianificare)

- Analisi preliminare del problema
- Analisi preliminare della codebase
- Brainstorming e Q&A esplorativo
- Scrittura specifica iniziale
- Generazione file Claude.md
- Generazione development-plan.md
- Raffinamento specifica

Claude Code

Fasi downstream (costruire, verificare)

- Utilizzo del file Claude.md e del development-plan.md come contesto
- Brainstorming con analisi codebase diretta
- Generazione piano di sviluppo dettagliato
- Implementazione controllata diff-by-diff
- Subagent-driven development
- Review dell'implementazione
- Integrazione e miglioramento file Claude.md
- Revisione e correzione iterativa

06

Context Engineering & Best Practices

Principi di ingegneria del contesto per l'AI

Context Engineering Best Practices



Modularità della documentazione

File separati per ogni concern (architettura, dominio, testing). Ciascun file rimane sotto i 5000 token.



Referencing esplicito tra file

Link tra documenti: "Per dettagli su architettura, consulta `.claude/architettura.md`". Bidirezionale.



Istruzioni "quando leggere"

"Leggi `sync.md` se lavori su sincronizzazione" — conferisce autonomia all'LLM.



Token budget bilanciato

Il file principale resta snello (300-500 token). I dettagli vivono nei file referenziati.



Enforcement rules

LOC limits, quality gates, self-check automatici. Claude diventa proattivo.



Aggiornamento bidirezionale

"Aggiorna `mappa-progetto.md` ogni volta che crei/sposti/elimini un file."

Componenti Essenziali dello SDD

Documenti Core



Project Overview

Scopo, contesto, obiettivi del progetto



Architecture

Stack, struttura, layer, pattern architetturali



Development Guidelines

Convenzioni, naming, code style, patterns



Testing Strategy

Cosa testare, come, copertura target

File Speciali



CLAUDE.md

Entry point: principi core + navigazione config



Enforcement Rules

LOC limits, quality gates, self-check automatici



Workflows

Development plan, project map, sprint tracking



Domain Knowledge

Terminologia, business rules, glossario dominio

Best Practices dallo SDD: Scelte Corrette

- Workflow strutturato in fasi con gate di approvazione
- Q&A prima dell'implementazione per eliminare ambiguità
- Piano salvato come artifact persistente (development-plan.md)
- Review come fase separata con specifica come benchmark

07

Il File CLAUDE.md

Entry point, principi core e workflow di generazione
(Lite, Intermediate, Advanced)

Cos'è il file CLAUDE.md?

CLAUDE.md è il "sistema operativo" del progetto per Claude Code. Rappresenta il primo file che Claude legge quando viene avviato e aperto in un progetto (folder), e ne caratterizza il contesto di partenza.



Scopo e contesto

Cosa facciamo, perchè, per chi.
Definisce il problema da risolvere.



Principi non negoziabili

Offline-first, zero data loss, validazione doppia.
Regole assolute del progetto.



Mappa documentazione

Struttura file + guida "quando leggere cosa"
per l'autonomia dell'LLM.



Istruzioni proattive

Monitoring LOC, refactoring automatico,
self-check obbligatori.

Anatomia del CLAUDE.md (Advanced Workflow)

Struttura File

```
CLAUDE.md      (entry point)
.claude/
  architettura.md (stack, layer, pattern)
  dominio.md      (business rules, glossario)
  qualita.md      (enforcement, checklist)
  sync.md         (offline-first, conflitti)
  testing.md      (strategia, copertura)
  sicurezza.md    (auth, RLS, OWASP)
docs/
  piano-sviluppo.md (sprint plan)
  mappa-progetto.md (file map)
  best-practices.md (standards universali)
```

Entry Point (300-500 token)

CLAUDE.md contiene i principi core e la navigazione verso i file modulari. Ogni concern vive nel suo file dedicato.

File Modulari (.claude/)

Ogni file copre un concern specifico. Claude carica solo ciò che serve, quando serve. Token budget ottimizzato.

Docs di Supporto

Piano di sviluppo, mappa progetto, best practices. Consultati on-demand. Non caricati all'avvio.

Referencing Bidirezionale

Ogni file dichiara le sue dipendenze e le condizioni per essere aggiornato. Claude non chiede permesso per leggere i file referenziati.

Autonomia di Claude: Leggere e Aggiornare

File	Leggilo quando...	Aggiornalo quando...
architettura.md	Crei nuovi file, componenti, hook, service	Cambia struttura directory o pattern
dominio.md	Implementi business logic, permessi, entità	Emergono nuove regole di business
qualita.md	Inizi qualsiasi task di sviluppo	Nuovi standard qualitativi necessari
sync.md	Lavori su changeLog, IndexedDB, Realtime	Cambia strategia di sync o conflict res.
testing.md	Scrivi o modifichi test, fai refactoring	Nuovi scenari di test identificati
sicurezza.md	Gestisci auth, RLS, input utente, API	Nuove policy sicurezza necessarie
piano-sviluppo.md	Pianifichi il prossimo sprint o task	Completamento task (segna stato)



**Claude non chiede il permesso per leggere i file referenziati: lo fa autonomamente.
Propone (e dopo approvazione esegue) aggiornamenti quando rileva drift tra configurazione e codice.**

Tre Workflow per la Generazione del CLAUDE.md

LITE

Setup: 2-4 ore
File: 6 file core
Fasi: 1 fase
Target: MVP, PoC, <1000 LOC

Flusso:

Features → Tech Stack → Genera CLAUDE.md

Generazione diretta: un prompt produce la configurazione. Veloce e pratico per progetti piccoli.

INTERMEDIATE

Setup: 4-8 ore
File: 8-10 file + 3 fasi
Fasi: 3 fasi
Target: Progetti medi, team 2-3

Flusso:

Config → Knowledge → Genera

Introduce analisi (Framework, Requirements) prima della generazione. Prompt più guidato.

ADVANCED

Setup: 1-2 giorni
File: 15+ file + 4 BP docs
Fasi: 3 + meta-prompting
Target: Production, team, long-term

Flusso:

Config → Knowledge + BP → Meta-prompt → Genera

Meta-prompting: genera prima il prompt ottimale, poi lo usa. Best Practices come input documentale.

LITE

Workflow Lite: Quick Start MVP

Struttura File

```
ShoppingListLite/  
  .claude/  
    CLAUDE.md  
    architecture.md  
    data-model.md  
    features-mvp.md  
    conventions.md  
    sync-strategy.md  
  prompts/  
    1_AskForFeatures.txt  
    2_AskForTechStacks.txt  
    3_GenerateConfig.txt
```

Caratteristiche

Setup

2-4 ore

File config

6 file core

Fasi

1 fase (generazione diretta)

Target

MVP, PoC, <1000 LOC

Flusso: Chiedi features --> Chiedi tech stack --> Genera configurazione CLAUDE.md

INTERMEDIATE

Workflow Intermediate: Progetto Claude + Analisi + Generazione Claude.md

Fase 1: Configuration

ProjectContext.md
ProjectDescription.txt
ProjectInstructions.md

Definizione problema e obiettivi

Fase 2: Knowledge

FrameworkAnalysis.md
SoftwareRequirements.md

Valutazione tecnologie e requisiti

Fase 3: ClaudeCode Config

CLAUDE.md + .claude/ (6 file)
Development Plan

Generazione configurazione completa



Differenza chiave rispetto al workflow Lite: introduce la fase di analisi (FrameworkAnalysis, SoftwareRequirements) prima della generazione. Il prompt di generazione del CLAUDE.md risulta più dettagliato e guidato.

Workflow Advanced: Generazione Claude.md tramite Meta-Prompting e Best Practices

1

Fase 1: Configuration

3 prompt sequenziali: Features, Write prompt, Generate

Meta-prompting: si genera prima il prompt ottimale, poi lo si usa per generare i documenti.

2

Fase 2: Knowledge + Docs

Best Practices come input documentale

4 documenti di best practices guidano la generazione:
Config, Enforcement, Modularization, Referencing.

3

Fase 3: ClaudeCode Config

CLAUDE.md + .claude/ + docs/ (10+ file)

Output: configurazione completa con enforcement rules, piano di sviluppo, mappa progetto, checklist QA.

Il Meta-Prompting nell'Advanced Workflow

Approccio Diretto (Lite)

"Genera il CLAUDE.md per questo progetto"

Un singolo prompt produce la configurazione finale.

Meta-Prompting (Advanced)

1. "Scrivi il miglior prompt per generare"
2. Usa quel prompt per generare il CLAUDE.md

Due passi: l'AI ottimizza prima le proprie istruzioni.

Best Practices come Input Documentale (Advanced Workflow)

Configuration Best Practices

Struttura CLAUDE.md, token budget, principi core

Enforcement Rules BP

LOC limits, quality gates, auto-refactoring triggers

Modularization Best Practices

Come suddividere in file modulari per concern

Referencing Best Practices

Link tra file, ancore, bidirezionalità dei riferimenti

Enforcement Rules: Claude Proattivo



File: max 200 LOC

A 150+ LOC, proponi refactoring automaticamente



Funzione: max 20 LOC

Estrai sotto-funzioni automaticamente se superato



TypeScript: strict, zero 'any'

Segnala violazioni prima del commit



Test coverage: target >80%

Suggerisci test mancanti per ogni service



Offline-first: DB locale = source of truth

Verifica che ogni operazione funzioni offline

Esempio di comportamento proattivo:

Claude: "Ho notato che questo file sta raggiungendo 150 LOC. Suggerisco di estrarre [funzionalità] in un modulo separato. Vuoi che proceda con il refactoring?"

Confronto Strutturale Side-by-Side

Aspetto	Lite	Intermediate	Advanced
File count	6 core	8-10 file + 3 fasi	10+ file + 4 BP docs
Setup time	2-4 ore	4-8 ore	1-2 giorni
Fasi workflow	1 (generazione)	3 (config/knowledge/code)	3 + meta-prompting
Analisi tech	Nessuna	FrameworkAnalysis	Framework + BP docs
Enforcement	Manuale	Semi-automatico	Automatico (LOC, test)
Meta-prompting	No	No	Si (prompt genera prompt)
Dev plan	No	Opzionale	Si (sprint plan completo)
Project map	No	No	Si (mappa ogni file)
Best for	PoC, MVP, <1000 LOC	Progetti medi, team 2-3	Production, team, long-term

Criteri Pratici di Scelta del Workflow

Usa LITE se:

- PoC o MVP da validare (<1000 LOC)
- 1-2 developer, no team distribuito
- Learning con Claude Code
- Budget tempo limitato, speed first

Usa INTERMEDIATE se:

- Progetti medi (1000-3000 LOC)
- Team 2-3 persone
- Serve analisi tecnologica preventiva
- Bilancio qualita/velocita

Usa ADVANCED se:

- Progetto production (>3000 LOC)
- Team 3+ o distribuito
- Requisiti complessi, long-term
- Enforcement automatico critico

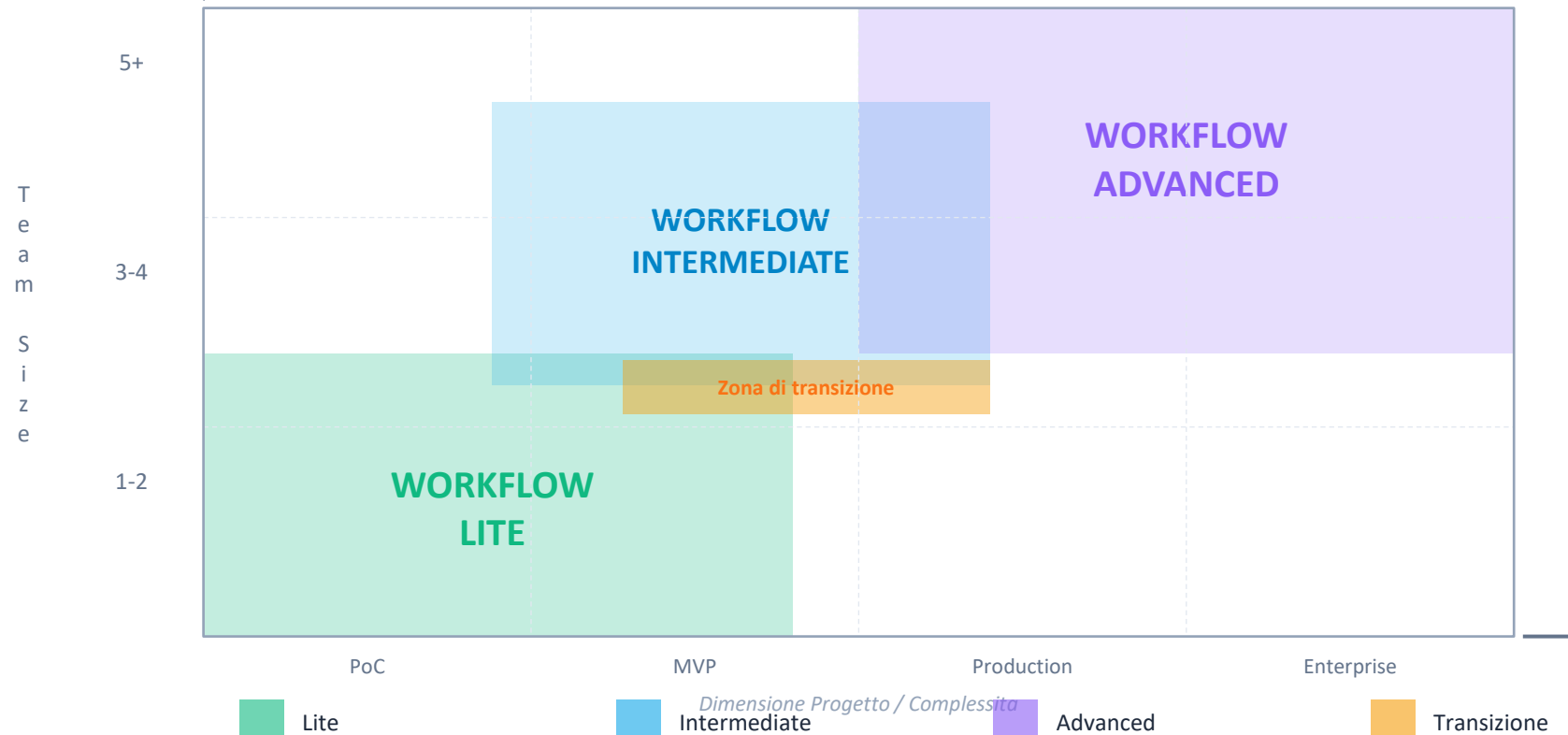
Percorso di Migrazione

Puoi iniziare con Lite e migrare verso Intermediate o Advanced quando il progetto cresce.

Segnali di migrazione: file >200 LOC, drift spec-codice, onboarding difficile, team >2 persone.

Non è una scelta irreversibile — i workflow sono complementari.

Matrice Decisionale



08

Prompt Templates per lo SDD

Template strutturati per ogni fase del workflow

Prompt Template: Claude Desktop (Generico)

```
# Task: [ID / NOME TASK]

## Contesto
[Background, Ruolo, Situazione, Obiettivo]

## Requisiti
1. [requisito specifico e misurabile]
2. [requisito specifico e misurabile]
3. [requisito specifico e misurabile]

## Formato
[Struttura, Lunghezza, Stile]

## Vincoli
[eventuali vincoli aggiuntivi]

## Esempi
[1-3 esempi del risultato desiderato]
```

Task

Identificativo chiaro del task

Contesto

Background che Claude non può inferire

Requisiti

Specifici, misurabili, numerati

Formato

Struttura dell'output desiderato

Vincoli

Limiti e regole aggiuntivi

Esempi

1-3 output di riferimento

I template strutturati garantiscono output coerenti e riducono le iterazioni necessarie.

Prompt Template: Claude Code - Planning Mode

```
# Task: [ID / NOME TASK]

## Obiettivo
[1-2 frasi: obiettivo e motivazioni]

## Requisiti
1. [requisito specifico e misurabile]
2. [requisito specifico e misurabile]

## Contesto
[info che Claude non deduce dalla codebase]

## Vincoli
[vincoli non già in CLAUDE.md]

--- Planning mode richiesto ---
1. leggi piano di sviluppo [development_plan.md]
2. genera piano di sviluppo dettagliato
3. richiedi approvazione del piano
```



Elemento Chiave

La sezione "Planning mode richiesto" obbliga Claude a leggere il piano, generare un piano dettagliato e chiedere approvazione PRIMA di scrivere codice.



Attenzione

Senza "Planning mode richiesto", Claude potrebbe iniziare a scrivere codice immediatamente senza una pianificazione.

Prompt Template: Claude Code - Brainstorming

```
# Task: [ID / NOME TASK]

## Obiettivo
[1-2 frasi: obiettivo e motivazioni]

## Requisiti
1. [requisito specifico e misurabile]
2. [requisito specifico e misurabile]

## Contesto
[info specifiche non deducibili dalla codebase]

## Vincoli
[requisiti non già in CLAUDE.md]
```



Trigger: **/brainstorming**

Separato dall'esecuzione per far emergere incognite e decisioni architetturali PRIMA di iniziare lo sviluppo.

Prompt Template: Claude Code - Advanced SDD (Esecuzione)

```
Planning mode richiesto:
1. leggi il piano di sviluppo [DEVELOPMENT_PLAN.md]
2. leggi la specifica e il piano di implementazione
3. genera un piano di sviluppo dettagliato
4. richiedi approvazione del piano
5. utilizza la skill superpowers:subagent-driven-development
```

09

Lezioni Apprese e Key Takeaways

Sintesi operativa e linee guida pratiche

Lezioni Apprese

1

Investire tempo nella specifica ripaga

Ogni ora di planning ne risparmia tre di correzioni. La copertura AI sale dal 60% al 95%.

2

Sfruttare il Q&A sistematicamente

Claude identifica gap che l'autore non vede. Il brainstorming è ancora più potente del semplice Q&A.

3

Essere espliciti sulle business rules

Se un comportamento non risulta scritto, Claude dovrà inventarlo. Le regole implicite generano bug.

4

Separare planning e implementazione

Usare Claude Desktop per pensare, Claude Code per costruire. Ogni strumento per il suo scopo.

5

Iterare sulla specifica, non solo sul codice

Migliorare il prompt è più efficace che correggere l'output. La spec è un documento vivo.

6

CLAUDE.md rappresenta il cuore dello SDD

Senza di esso, Claude Code opera senza una guida progettuale. Definisce contesto, principi e navigazione.

7

Modularizzazione = efficienza token

File separati per concern specifico. Claude carica solo ciò che serve, quando serve.

8

Enforcement rules rendono Claude proattivo

Monitoring LOC, qualità del codice e test coverage diventano automatici.

Key Takeaways: Mappa Concettuale SDD



***La qualità del risultato
è proporzionale alla qualità
della specifica.***

Spec-Driven Development: non è vibe coding.

Stefano Zaghi | stefano.zaghi@gamma-spa.com

Corso Claude Desktop & Claude Code | Gamma SPA